Beacon AI

Developer Manual — Install, Run, and Extend

A voice-controlled, AI-powered accessibility browser extension for Chrome, built for blind and low-vision users.

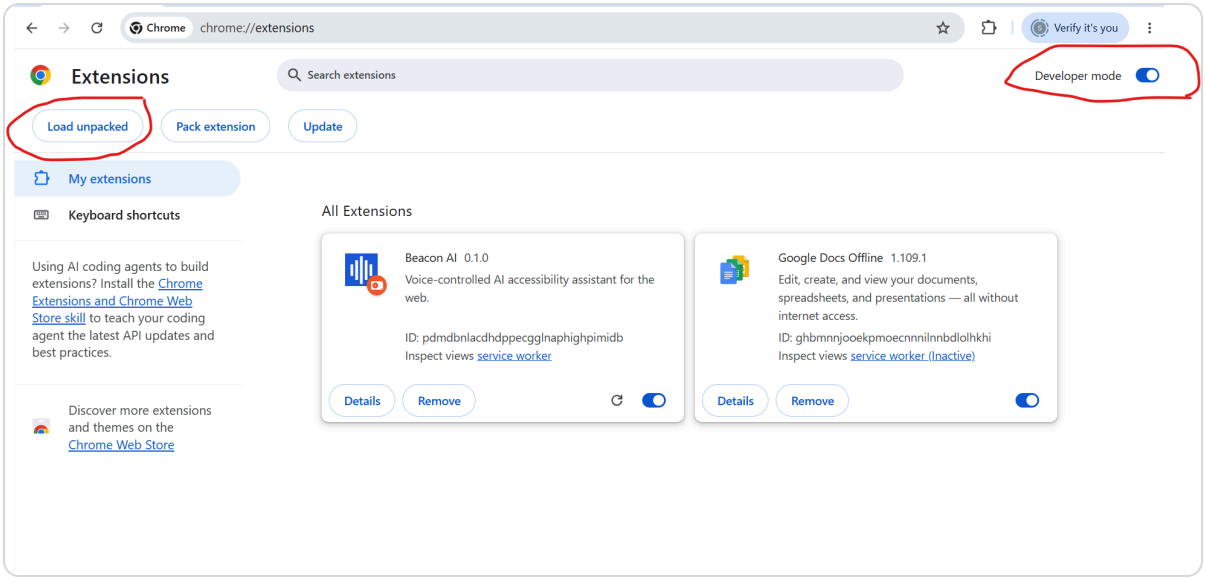
1. What's in this repo

```
Beacon/
  extension/       — the Chrome extension itself (Manifest V3)
  backend/         — FastAPI backend that calls the Claude API (deployed to Railway)
  docs/            — this manual, the user manual, and screenshots
  PROJECT_PLAN.md  — full product spec, design principles, open questions
```

2. Quickest way to try it (no backend setup needed)

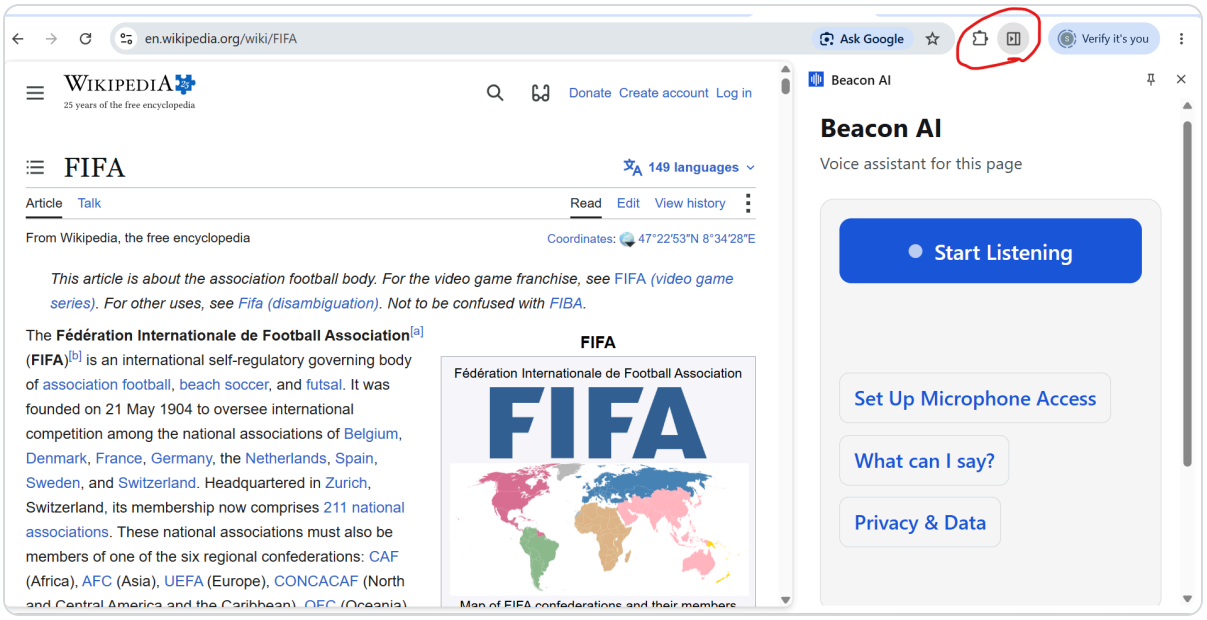
The extension already points at a **deployed backend** on Railway, so nothing needs to run locally just to try Beacon AI out.

1. Open Chrome and go to `chrome://extensions`
2. Turn on **Developer mode** (top-right toggle)
3. Click **Load unpacked**
4. Select the `extension/` folder from this repo



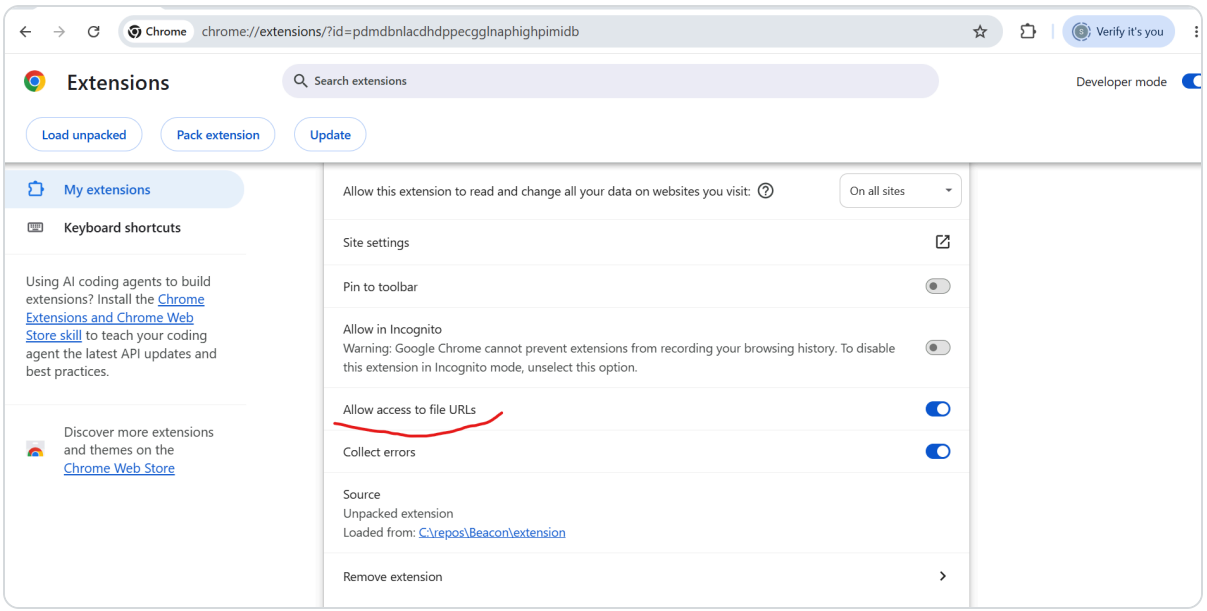
Developer mode + Load unpacked, on the chrome://extensions page.

5. Pin the extension (puzzle-piece icon in the toolbar → pin Beacon AI)
6. Click the Beacon AI icon — it opens a **side panel**, not a popup



The main panel, open alongside a page.

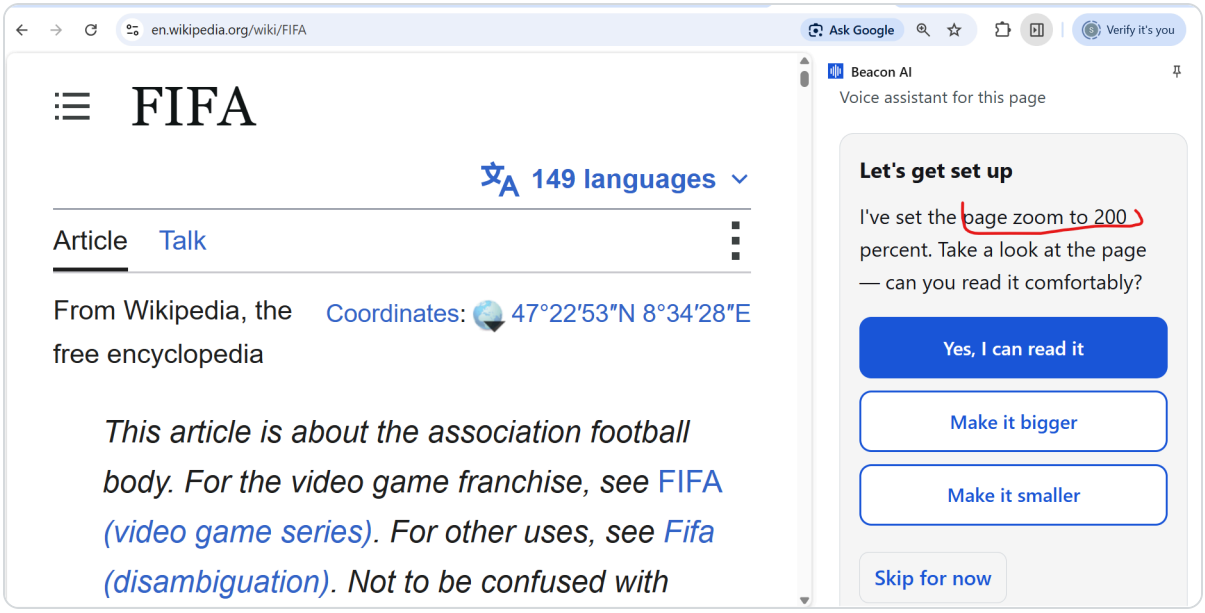
> **Allow local file access** (needed for the PDF-reading feature): go to `chrome://extensions`, find Beacon AI → **Details**, and toggle **"Allow access to file URLs"** on.



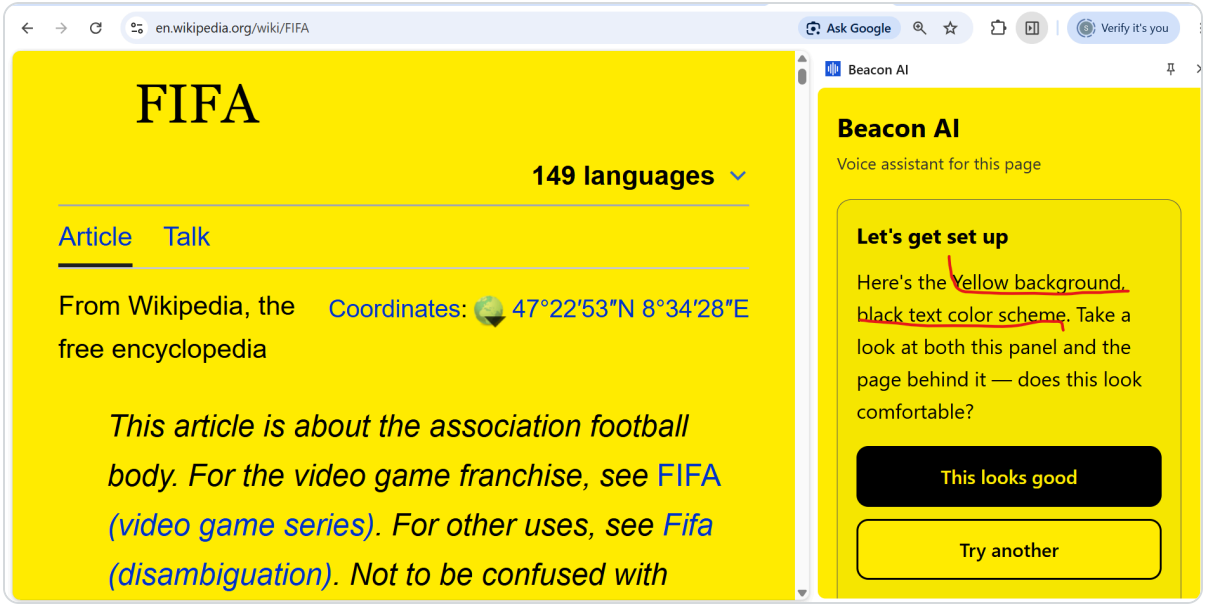
This setting is easy to miss — it lives on the extension's Details page.

First run

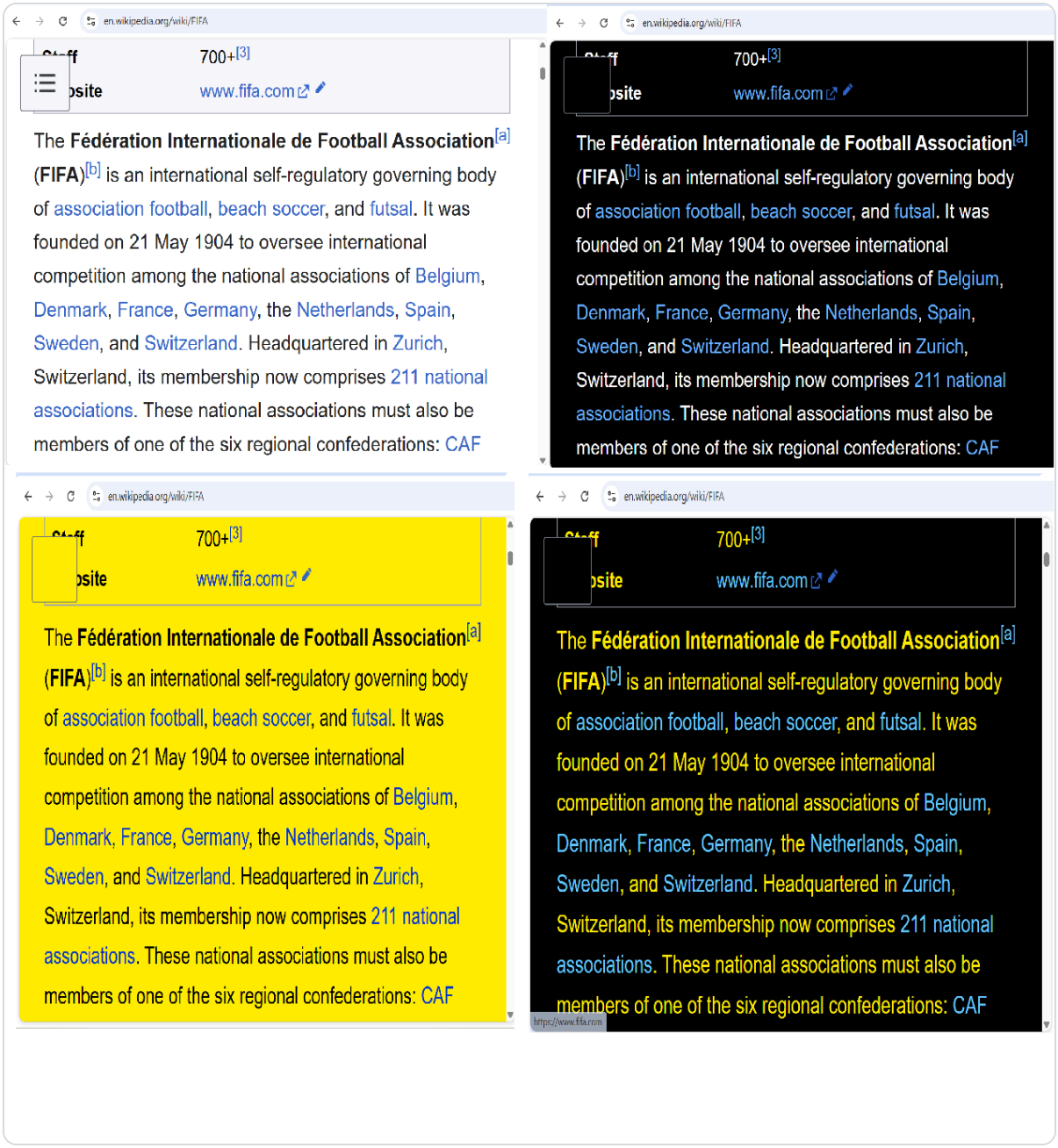
The panel walks through a short setup automatically the first time: speech rate, page zoom, and contrast theme, each calibrated by trying it live and confirming it's comfortable. Redo this any time — say "*redo setup*" or click **Redo Setup** in the panel.



Zoom calibration applies a candidate zoom to the real page, not just a description.



Theme calibration previews live on both the panel and the page at once.



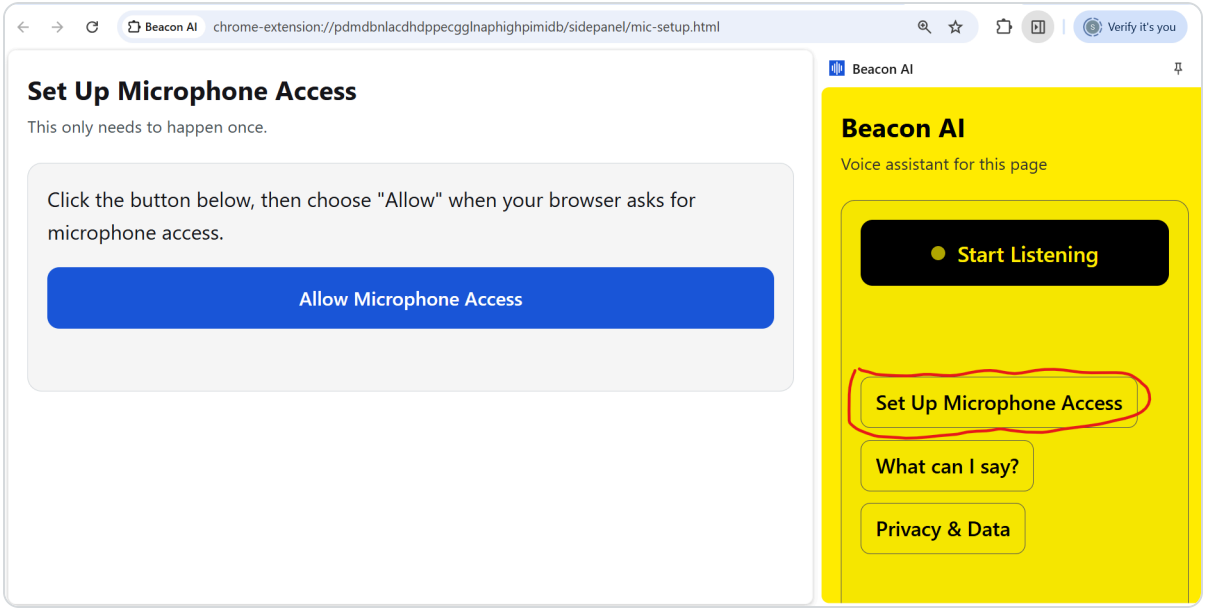
Once chosen, a theme applies to any page you read — not just the panel. Clockwise from top-left: light, dark, black-yellow, yellow-black.

Microphone permission (one-time, Chrome-specific quirk)

Chrome doesn't reliably prompt for microphone access from inside a side panel — a known Chrome limitation, not a bug in the extension. If voice input doesn't work the first time:

1. Click **"Set Up Microphone Access"** in the panel

2. A new tab opens with one button — click **"Allow Microphone Access"**
3. Approve the browser's permission prompt in that tab
4. Close the tab and return to the side panel — it should now work



A single-purpose page avoids the ambiguity of reusing the full panel UI for this step.

Try it

Say "help" or click **"What can I say?"** in the panel for a categorized command list. Quick starting points:

- "*summarize this page*", "*give me the key points*", "*read this page*"
- "*zoom in*", "*scroll down*", "*go back*"
- "*translate this to Spanish*", "*define [some word]*"
- "*speak faster*" / "*speak slower*"
- "*dark theme*", "*yellow on black*"
- "*privacy*" — what happens to your data

3. Running the backend locally

Only needed if you're changing backend code — the deployed Railway backend works out of the box otherwise.

```
cd backend
python -m venv .venv

# Windows
.venv\Scripts\activate
# macOS/Linux
source .venv/bin/activate

pip install -r requirements.txt
cp .env.example .env   # then edit .env and add your own ANTHROPIC_API_KEY

uvicorn main:app --host 127.0.0.1 --port 8000
```

Then point the extension at your local backend — in `extension/background.js`, change:

```
const BACKEND_URL = "https://beacon-ai-production-4999.up.railway.app";
```

to:

```
const BACKEND_URL = "http://localhost:8000";
```

...and reload the extension. Switch it back before committing.

Sanity check: `curl http://127.0.0.1:8000/health`

4. Running the tests

The intent-matching logic (`extension/lib/commands.js` — every voice command's phrase matching) has a plain Node test suite, no browser needed:

```
cd extension
node --test test/commands.test.js
```

There's no automated test suite for the rest of the extension (browser-side voice/AI/UI behavior) — that's verified by hand in Chrome, the appropriate surface for it.

5. Project structure

Extension

```
extension/
  manifest.json          — MV3 config, permissions, side panel registration
  background.js           — service worker: tab commands (zoom/scroll/nav), AI
backend calls,
                             page-content/PDF extraction caching, ambient zoom +
contrast theme
  content.js               — injected into pages on demand; runs Readability.js
extraction
  lib/
    commands.js              — voice -> intent matching (regex-based), fully unit
tested
    pageCache.js              — chrome.storage.session-backed extraction cache
    Readability.js            — vendored from @mozilla/readability
    pdf.min.mjs, pdf.worker.min.mjs — vendored pdf.js, used for PDF text extraction
  sidepanel/
    sidepanel.html/js/css      — the main UI: voice controls, onboarding, settings
    mic-setup.html/js           — the dedicated one-button mic permission page
  test/
    commands.test.js            — intent-matcher unit tests
```

Backend

```
backend/
  main.py             — FastAPI app; single /query endpoint, Claude API call with
prompt caching
  requirements.txt
  Procfile            — Railway start command
  .env.example        — copy to .env locally, fill in ANTHROPIC_API_KEY
```

6. Known limitations

- Chrome/Edge only (Manifest V3), no other browsers
- TTS quality for non-English languages depends on what voices your OS/Chrome install has available — if none exist for a requested language, Beacon says so honestly rather than mangling the audio
- Page-content contrast theming targets text-heavy pages; complex web apps/dashboards may render oddly when overridden
- No automated end-to-end test suite — voice/AI/UI behavior is verified manually in Chrome

See `PROJECT_PLAN.md` in the repo root for the full product spec, design principles, and open questions.